Django Complete Notes (A to Z)
Complete Backend Developer Study Guide

Language: English
Framework: Django + Django REST Framework
Database: PostgreSQL / SQLite
Level: Beginner to Advanced

---

1. Django Fundamentals

1.1 What is Django?

Django is a high-level, open-source Python web framework used to build secure, scalable, and maintainable web applications quickly.

Django follows the MVT (Model-View-Template) architectural pattern.

Main Features of Django

1. Fast Development
2. Built-in Admin Panel
3. ORM (Object Relational Mapping)
4. Authentication System
5. URL Routing
6. Middleware
7. Security Protection
8. Form Handling
9. Database Migrations
10. Session Management
11. Template Engine
12. Reusable Applications
13. Scalable Architecture
14. Built-in Testing Tools

1.2 MVT Architecture

MVT stands for:

- **Model:** Handles database structure and data.
- **View:** Handles business logic and request processing.
- **Template:** Handles frontend presentation.

MVT Workflow

```text
User
  |
  v
URL Dispatcher
  |
  v
View
  |
  +------> Model ------> Database
  |
  v
Template
  |
  v
HTTP Response
  |
```

```
  v
User
```

Example

Model:

```python
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return self.name
```

View:

```python
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.all()

    return render(
        request,
        "students/list.html",
        {"students": students}
    )
```

Template:

```html
<h1>Student List</h1>

{% for student in students %}
    <p>{{ student.name }} - {{ student.age }}</p>
{% empty %}
    <p>No students found.</p>
{% endfor %}
```

1.3 Django vs Flask vs FastAPI

Feature	Django	Flask	FastAPI
Type	Full-stack framework	Micro framework	API framework
Language	Python	Python	Python
ORM	Built-in	External	External
Admin Panel	Built-in	External	External
Authentication	Built-in	Extensions	External
API Development	DRF	Extensions	Built-in support
Async Support	Supported	Supported	Excellent
Best For	Full web apps	Small apps	Modern APIs

When to use Django?

- E-commerce applications

- Management systems
- Admin dashboards
- Blog applications
- Large backend applications
- REST API projects using DRF

1.4 Django Project vs Django App

Project: The complete Django website configuration.

App: A reusable module that performs a specific function.

Example:

```text
ecommerce/
│
├── config/
├── users/
├── products/
├── orders/
├── payments/
└── manage.py
```

Here:

- `ecommerce` is the project.
- `users`, `products`, `orders`, `payments` are apps.

One Django project can contain multiple apps.

1.5 Request-Response Lifecycle

When a user sends a request:

1. Browser sends HTTP request.
2. Django receives the request.
3. Request middleware executes.
4. URL resolver finds the matching URL.
5. Django calls the corresponding view.
6. View executes business logic.
7. Model communicates with the database if needed.
8. View creates an HTTP response.
9. Response middleware executes.
10. Django returns the response to the browser.

1.6 WSGI and ASGI

WSGI

WSGI = Web Server Gateway Interface.

It is a standard interface between a Python web application and a web server.

Traditionally used for synchronous Django applications.

ASGI

ASGI = Asynchronous Server Gateway Interface.

It supports asynchronous applications and protocols such as WebSockets.

WSGI vs ASGI

WSGI	ASGI
Traditional synchronous interface	Supports synchronous and asynchronous applications
HTTP request-response	HTTP and asynchronous protocols
Does not provide native WebSocket support	Supports WebSockets
Commonly used with Gunicorn	Commonly used with Uvicorn or Daphne

1.7 Middleware

Middleware is a component that processes requests before they reach the view and responses before they return to the client.

Built-in Middleware

- SecurityMiddleware
- SessionMiddleware
- CommonMiddleware
- CsrfViewMiddleware
- AuthenticationMiddleware
- MessageMiddleware
- XFrameOptionsMiddleware

Middleware Execution

Request:

```text
Request
  |
SecurityMiddleware
  |
SessionMiddleware
  |
CommonMiddleware
  |
CSRF Middleware
  |
AuthenticationMiddleware
  |
View
```

Response travels back through middleware in reverse order.

Custom Middleware

```python
class SimpleMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):

        print("Before View")

        response = self.get_response(request)

        print("After View")

        return response
```

Register:

```python
MIDDLEWARE = [
    "myapp.middleware.SimpleMiddleware",
]
```

Important: Middleware order matters because middleware can depend on other middleware.

---

2. Setup and Project Structure

2.1 Install Python

Check Python:

```bash
python --version
```

Check pip:

```bash
pip --version
```

2.2 Create Virtual Environment

```bash
python -m venv venv
```

Activate on Windows

```bash
venv\Scripts\activate
```

Activate on Linux/macOS

```bash
source venv/bin/activate
```

Deactivate:

```bash
deactivate
```

Why virtual environment?

A virtual environment keeps project dependencies isolated from other Python projects.

2.3 Install Django

```bash
pip install django
```

Check version:

```bash
django-admin --version
```

Install Django REST Framework:

```bash
pip install djangorestframework
```

2.4 Create Django Project

```bash
django-admin startproject config .
```

The dot creates the project in the current directory.

2.5 Create Django App

```bash
python manage.py startapp students
```

2.6 Register Apps

Inside `settings.py`:

```python
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "rest_framework",

    "students",
]
```

You can also register an AppConfig:

```python
"students.apps.StudentsConfig"
```

2.7 Run Development Server

```bash
python manage.py runserver
```

Custom port:

```bash
python manage.py runserver 8001
```

The development server is for development, not production.

2.8 Django Project Structure

```text
project/
│
├── manage.py
│
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── students/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
│
├── templates/
├── static/
├── media/
├── requirements.txt
└── venv/
```

Important Files

File	Purpose
manage.py	Runs management commands
settings.py	Project configuration
urls.py	URL routing
views.py	Request handling and business logic
models.py	Database models
admin.py	Admin panel configuration
apps.py	Application configuration
tests.py	Automated tests
migrations/	Database migration files
wsgi.py	WSGI application
asgi.py	ASGI application

manage.py

Used to execute Django commands.

```bash
python manage.py runserver
python manage.py migrate
python manage.py createsuperuser
```

settings.py

Contains project configuration such as:

- Installed apps
- Middleware

- Database
- Templates
- Static files
- Security
- Time zone
- Secret key

urls.py

Maps URLs to views.

views.py

Contains request-handling logic.

models.py

Defines database tables and relationships.

admin.py

Registers and customizes models in Django Admin.

apps.py

Contains application configuration.

tests.py

Contains automated tests.

2.9 Environment Variables and Configuration

Environment variables keep sensitive configuration outside source code.

Install:

```bash
pip install python-dotenv
```

Create `.env`:

```env
SECRET_KEY=your-secret-key
DEBUG=True
DATABASE_NAME=mydatabase
DATABASE_USER=postgres
DATABASE_PASSWORD=your-password
```

Load variables:

```python
import os
from dotenv import load_dotenv

load_dotenv()

SECRET_KEY = os.getenv("SECRET_KEY")

DEBUG = os.getenv("DEBUG", "False") == "True"
```

Add `.env` to `.gitignore`.

Never commit production secrets to GitHub.

---

3. URLs, Views, and Responses

3.1 URL Routing

URL routing maps a URL to a Django view.

Project `urls.py`:

```python
from django.contrib import admin
from django.urls import path

from students import views

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", views.home, name="home"),
]
```

View:

```python
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django")
```

3.2 Path Converters

Path converters capture values from URLs.

```python
from django.urls import path

urlpatterns = [
    path("student/<int:id>/", views.student_detail),
]
```

View:

```python
def student_detail(request, id):
    return HttpResponse(f"Student ID: {id}")
```

Built-in Converters

Converter	Description
str	Non-empty string excluding `/`
int	Positive integer
slug	Letters, numbers, hyphens and underscores
uuid	UUID value
path	Non-empty path including `/`

3.3 Include App URLs

Project `urls.py`:

```python
from django.urls import path, include

urlpatterns = [
    path("students/", include("students.urls")),
]
```

App `students/urls.py`:

```python
from django.urls import path
from . import views

urlpatterns = [
    path("", views.student_list, name="student_list"),
    path("<int:id>/", views.student_detail, name="student_detail"),
]
```

3.4 Function-Based Views (FBV)

A Function-Based View is a Python function that receives a request and returns a response.

```python
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Django")
```

POST example:

```python
def create_student(request):

    if request.method == "POST":
        return HttpResponse("Student Created")

    return HttpResponse("Send POST request")
```

3.5 Class-Based Views (CBV)

A Class-Based View uses a Python class to handle requests.

```python
from django.views import View
from django.http import HttpResponse

class HomeView(View):

    def get(self, request):
        return HttpResponse("GET Request")

    def post(self, request):
        return HttpResponse("POST Request")
```

URL:

```python
path("", HomeView.as_view(), name="home")
```

FBV vs CBV

FBV	CBV
Uses functions	Uses classes
Simple for small views	Useful for reusable views
Explicit control flow	Supports inheritance and mixins
Easy to understand initially	Useful for structured applications

3.6 HttpRequest

The request object contains information about an incoming request.

Important properties:

```python
request.method
request.GET
request.POST
request.FILES
request.user
request.path
request.headers
request.COOKIES
request.session
```

Example:

```python
def search(request):
    keyword = request.GET.get("q", "")

    return HttpResponse(f"Search: {keyword}")
```

URL:

```text
/search/?q=django
```

3.7 HttpResponse

Returns an HTTP response.

```python
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello World")
```

3.8 render()

Renders an HTML template.

```python
```

```python
from django.shortcuts import render

def home(request):

    context = {
        "name": "Faruk",
        "age": 24
    }

    return render(request, "home.html", context)
```

3.9 redirect()

Redirects the user to another URL.

```python
from django.shortcuts import redirect

def home(request):
    return redirect("student_list")
```

3.10 JsonResponse()

Returns JSON data.

```python
from django.http import JsonResponse

def api(request):

    data = {
        "name": "Faruk",
        "role": "Backend Developer"
    }

    return JsonResponse(data)
```

For a list:

```python
return JsonResponse(
    [{"name": "Faruk"}, {"name": "Omar"}],
    safe=False
)
```

3.11 404 and 500 Error Handling

404 means the requested resource was not found.

```python
from django.shortcuts import get_object_or_404

def student_detail(request, id):

    student = get_object_or_404(Student, id=id)

    return render(
        request,
        "detail.html",
        {"student": student}
```

```
    )
```

Custom 404 handler:

```python
handler404 = "myapp.views.custom_404"
```

View:

```python
from django.shortcuts import render

def custom_404(request, exception):
    return render(
        request,
        "404.html",
        status=404
    )
```

500 means an internal server error occurred.

Custom handler:

```python
handler500 = "myapp.views.custom_500"
```

Custom error pages are normally tested with `DEBUG = False`.

3.12 Named URLs and reverse()

Named URLs make URL management easier.

```python
path(
    "students/",
    views.student_list,
    name="student_list"
)
```

Python:

```python
from django.urls import reverse

url = reverse("student_list")
```

Template:

```html
<a href="{% url 'student_list' %}">Students</a>
```

3.13 Query Parameters vs Path Parameters

Path Parameter

```text
/students/10/
```

```

```python
path("students/<int:id>/", views.detail)
```

Query Parameter

```text
/students/?age=20
```

```python
age = request.GET.get("age")
```

Path Parameter	Query Parameter
Part of URL path	Comes after `?`
Identifies a resource	Often used for filtering/search
`/users/10/`	`/users/?age=20`

---

4. Templates

4.1 What is a Template?

A Django template is an HTML file that displays dynamic data.

Example:

```html
<h1>Welcome {{ name }}</h1>
```

4.2 Template Configuration

Inside `settings.py`:

```python
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

4.3 Template Inheritance

Inheritance allows multiple templates to reuse a common layout.

Base template:

```html

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>

    <nav>My Website</nav>

    {% block content %}
    {% endblock %}

</body>
</html>
```

Child template:

```html
{% extends "base.html" %}

{% block title %}
Home Page
{% endblock %}

{% block content %}
<h1>Welcome to Django</h1>
{% endblock %}
```

4.4 Template Variables

```html
<h1>{{ name }}</h1>
<p>{{ age }}</p>
```

4.5 Template Filters

Filters modify displayed values.

```html
{{ name|lower }}

{{ name|upper }}

{{ name|title }}

{{ name|length }}

{{ name|default:"Unknown" }}

{{ description|truncatewords:10 }}

{{ date|date:"Y-m-d" }}
```

4.6 Template Tags

Tags provide logic and functionality.

If:

```html
```

```
{% if age >= 18 %}
    <p>Adult</p>
{% else %}
    <p>Minor</p>
{% endif %}
```

For:

```html
{% for student in students %}
    <p>{{ student.name }}</p>
{% empty %}
    <p>No students found</p>
{% endfor %}
```

4.7 Template Context

Context is a dictionary of values passed from a view to a template.

```python
def home(request):

    context = {
        "name": "Faruk",
        "skills": ["Python", "Django", "DRF"]
    }

    return render(request, "home.html", context)
```

Template:

```html
<h1>{{ name }}</h1>

{% for skill in skills %}
    <p>{{ skill }}</p>
{% endfor %}
```

4.8 Static Files

Static files include CSS, JavaScript, images, and fonts.

Settings:

```python
STATIC_URL = "static/"

STATICFILES_DIRS = [
    BASE_DIR / "static"
]

STATIC_ROOT = BASE_DIR / "staticfiles"
```

Template:

```html
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

```

## 4.9 Forms in Templates

```html
<form method="POST">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Submit</button>
</form>
```

4.10 CSRF Protection

CSRF = Cross-Site Request Forgery.

Django uses CSRF tokens to protect unsafe form submissions.

```html
<form method="POST">
    {% csrf_token %}
    <button type="submit">Submit</button>
</form>
```

4.11 Autoescaping and Safe Rendering

Django automatically escapes many HTML-sensitive characters in template variables.

Example:

```html
{{ user_input }}
```

This helps prevent XSS attacks.

Avoid:

```html
{{ user_input|safe }}
```

when the value comes from an untrusted user.

Use `safe` only when you intentionally trust and sanitize the HTML.

---

5. Models and Database

5.1 What is a Model?

A Django model is a Python class that represents a database table.

Each model field generally represents a database column.

```python
from django.db import models

class Student(models.Model):

```
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return self.name
```

5.2 Field Types

Field	Purpose
CharField	Short text
TextField	Long text
IntegerField	Integer
PositiveIntegerField	Positive integer
BigIntegerField	Large integer
FloatField	Floating-point number
DecimalField	Precise decimal
BooleanField	True/False
DateField	Date
TimeField	Time
DateTimeField	Date and time
EmailField	Email
URLField	URL
SlugField	Slug
UUIDField	UUID
FileField	File upload
ImageField	Image upload
JSONField	JSON data
ForeignKey	Many-to-one
OneToOneField	One-to-one
ManyToManyField	Many-to-many

5.3 Field Options

- `null=True`: Allows NULL in the database.
- `blank=True`: Allows empty input during validation.
- `default`: Default value.
- `unique=True`: Unique values.
- `primary_key=True`: Primary key.
- `db_index=True`: Creates an index.
- `editable=False`: Excludes field from model forms.
- `choices`: Predefined choices.
- `verbose_name`: Human-readable name.
- `help_text`: Help information.
- `max_length`: Maximum string length.

Example:

```python
class Product(models.Model):

    name = models.CharField(
        max_length=100,
        unique=True
    )

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        default=0
    )
```

```
    is_active = models.BooleanField(default=True)
```

null vs blank

null	blank
Database-level option	Validation-level option
Controls NULL storage	Controls empty input

For optional text fields, Django commonly recommends `blank=True` without `null=True` to avoid two representations of empty text.

5.4 Primary Keys

A primary key uniquely identifies a database record.

Django automatically creates a primary key if you do not define one.

```python
id = models.AutoField(primary_key=True)
```

UUID primary key:

```python
import uuid

id = models.UUIDField(
    primary_key=True,
    default=uuid.uuid4,
    editable=False
)
```

5.5 Unique Constraints

Single-field uniqueness:

```python
email = models.EmailField(unique=True)
```

Multi-field uniqueness:

```python
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["first_name", "last_name"],
            name="unique_full_name"
        )
    ]
```

5.6 ForeignKey

ForeignKey creates a many-to-one relationship.

```python
class Department(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):
```

```
    name = models.CharField(max_length=100)

    department = models.ForeignKey(
        Department,
        on_delete=models.CASCADE,
        related_name="students"
    )
```

Many students can belong to one department.

on_delete Options

- `CASCADE`: Deletes dependent records.
- `PROTECT`: Prevents deletion.
- `SET_NULL`: Sets the relationship to NULL.
- `SET_DEFAULT`: Sets the default.
- `SET(value)`: Sets a specific value.
- `DO_NOTHING`: Takes no automatic action; database constraints may still reject deletion.

5.7 OneToOneField

Creates a one-to-one relationship.

```python
class Profile(models.Model):

    student = models.OneToOneField(
        Student,
        on_delete=models.CASCADE,
        related_name="profile"
    )

    phone = models.CharField(max_length=20)
```

One student has one profile.

5.8 ManyToManyField

Creates a many-to-many relationship.

```python
class Course(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):

    name = models.CharField(max_length=100)

    courses = models.ManyToManyField(
        Course,
        related_name="students"
    )
```

One student can enroll in many courses, and one course can have many students.

5.9 Model Methods

Model methods define custom behavior.

```python
```

```
class Student(models.Model):

    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def is_adult(self):
        return self.age >= 18

    def __str__(self):
        return self.name
```

5.10 __str__()

`__str__()` returns a readable string representation of an object.

```python
def __str__(self):
    return self.name
```

Without a custom `__str__()`, objects may appear with less useful default representations in the admin and shell.

5.11 class Meta

`class Meta` is an inner class inside a Django model. It defines metadata and configuration options.

Example:

```python
class Student(models.Model):

    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        db_table = "students"
        ordering = ["name"]
        verbose_name = "Student"
        verbose_name_plural = "Students"

    def __str__(self):
        return self.name
```

Important Meta Options

Option	Meaning
db_table	Custom database table name
ordering	Default ordering
verbose_name	Singular display name
verbose_name_plural	Plural display name
abstract	Abstract model
proxy	Proxy model
managed	Django table management
indexes	Database indexes
constraints	Database constraints
unique_together	Legacy multi-field uniqueness
default_related_name	Default reverse relation name
get_latest_by	Default latest-object field
permissions	Custom permissions

| app_label | Explicit application label |

ordering

```python
class Meta:
    ordering = ["name"]
```

Descending:

```python
class Meta:
    ordering = ["-created_at"]
```

db_table

```python
class Meta:
    db_table = "student_table"
```

verbose_name

```python
class Meta:
    verbose_name = "Student"
    verbose_name_plural = "Students"
```

indexes

```python
class Meta:
    indexes = [
        models.Index(fields=["name"]),
    ]
```

constraints

```python
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["name", "age"],
            name="unique_name_age"
        )
    ]
```

abstract model

```python
class BaseModel(models.Model):

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

class Student(BaseModel):
```

```
    name = models.CharField(max_length=100)
```

proxy model

```python
class StudentProxy(Student):

    class Meta:
        proxy = True
        ordering = ["name"]
```

managed

```python
class Meta:
    managed = False
    db_table = "existing_student_table"
```

`managed=False` tells Django not to manage the table's creation and deletion through normal migrations.

Important: `class Meta` is not itself a model. It configures the model.

5.12 Introspection
1. What is Introspection?

Introspection is the process of examining an object, class, or program at runtime to discover information about its structure, attributes, methods, and type.

In Python, introspection helps us understand:

Which class an object belongs to.

Which attributes an object has.

Which methods are available.

What type of object we are working with.

Django also provides introspection features to inspect model metadata and database structures.

2. Understanding __class__ in Python

__class__ is a built-in Python attribute that returns the class of an object.

Example

```
class Student:
    pass

student = Student()

print(student.__class__)
print(student.__class__.__name__)
```

Output:

```
<class '__main__.Student'>
Student
```

Explanation

student.__class__ returns the class object of student.

student.__class__.__name__ returns the class name as a string.

Here, the object's class is Student.

3. Understanding the Meta Class

In Django, Meta is an inner class used inside a model to define model-level configuration.

It can define:

Database table names.

Default ordering.

Verbose names.

Unique constraints.

Database indexes.

Permissions.

Example

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        db_table = "students"
        ordering = ["name"]
        verbose_name = "Student"
        verbose_name_plural = "Students"
```

Explanation

Meta is an inner class.

db_table specifies the database table name.

ordering defines the default ordering of query results.

verbose_name defines the human-readable singular name.

verbose_name_plural defines the human-readable plural name.

4. Understanding Meta.__class__

Since Meta is also a Python class, we can inspect its type using __class__.

Example

```
class Student:
    class Meta:
        ordering = ["name"]
```

```
print(Student.Meta.__class__)
print(Student.Meta.__class__.__name__)
```

Output:

```
<class 'type'>
type
```

Explanation

Student.Meta refers to the inner Meta class.

Student.Meta.__class__ returns its class.

In Python, classes are instances of type.

Therefore, the result is type.

Important: Meta.__class__ and student.__class__ are not the same.

student.__class__ returns the class of the object.

Student.Meta.__class__ returns the metaclass of the Meta class, which is normally type.

5. Django Model Meta vs _meta

These two concepts are related but different.

Meta	_meta
Inner class inside a model	Django-generated model metadata API
Defines model configuration	Provides access to model metadata
Written by the developer	Created and managed by Django
Example: db_table	Example: model._meta.db_table

Example

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        db_table = "students"
        ordering = ["name"]
```

Accessing model metadata:

```
print(Student._meta.db_table)
```

```
print(Student._meta.ordering)
print(Student._meta.model_name)
print(Student._meta.app_label)
```

Possible Output:

```
students
['name']
student
your_app_name
```

The actual app_label depends on your Django application.

Explanation

Student.Meta is the inner configuration class.

Student._meta is Django's model metadata interface.

_meta provides information about the model, fields, relationships, and database configuration.

6. Using _meta for Introspection

Django's _meta API allows us to inspect model information dynamically.

6.1 Get the Model Name

```
print(Student._meta.model_name)
```

6.2 Get the Database Table Name

```
print(Student._meta.db_table)
```

6.3 Get the Application Name

```
print(Student._meta.app_label)
```

6.4 Get All Model Fields

```
for field in Student._meta.get_fields():
    print(field.name)
```

This returns the model's fields and may also include relationship fields, depending on the model structure.

6.5 Get a Specific Field

```
field = Student._meta.get_field("name")

print(field.name)
print(field.get_internal_type())
print(field.max_length)
```

Possible Output:

```
name
CharField
100
```

6.6 Check Whether a Field Is Primary Key

```
field = Student._meta.get_field("id")
```

print(field.primary_key)

Output:

True

7. Practical Use Cases of Introspection

Introspection is useful when building:

Dynamic serializers.

Generic API systems.

Reusable Django applications.

Automatic form generation.

Model inspection tools.

Admin customization.

Generic reporting systems.

For example, instead of manually writing every model field, we can inspect model metadata and generate information dynamically.

Example:

```python
from django.db import connection

with connection.cursor() as cursor:

    tables = connection.introspection.table_names(
        cursor
    )

print(tables)
```

Get table description:

```python
with connection.cursor() as cursor:

    description = connection.introspection.get_table_description(
        cursor,
        "students_student"
    )

print(description)
```

Django management command:

```bash
python manage.py inspectdb
```

`inspectdb` examines an existing database and generates model definitions.

5.13 Migrations

Migrations manage database schema changes.

Create migration:

```bash
python manage.py makemigrations
```

Apply migration:

```bash
python manage.py migrate
```

Show migrations:

```bash
python manage.py showmigrations
```

Show SQL:

```bash
python manage.py sqlmigrate students 0001
```

Migration Workflow

1. Create or update models.
2. Run `makemigrations`.
3. Review the migration.
4. Run `migrate`.
5. Verify the database.

5.14 Database Configuration

SQLite configuration:

```python
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

PostgreSQL configuration:

```python
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydatabase",
        "USER": "myuser",
        "PASSWORD": "mypassword",
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

Install driver:

```bash
pip install "psycopg[binary]"
```

Use environment variables for credentials.

5.15 Django Admin Registration

```python
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

Admin Customization

```python
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):

    list_display = ("id", "name", "age")
    list_filter = ("age",)
    search_fields = ("name",)
    ordering = ("name",)
    list_per_page = 20
```

Create superuser:

```bash
python manage.py createsuperuser
```

---

6. ORM and QuerySets

6.1 Database Basics

A database stores and manages structured data.

Important terms:

- Database: Collection of data.
- Table: Collection of records.
- Row: One record.
- Column: One field.
- Primary Key: Unique record identifier.
- Foreign Key: Relationship between tables.
- Index: Helps speed up queries.
- Constraint: Enforces data rules.
- Transaction: Group of database operations.

6.2 What is ORM?

ORM = Object Relational Mapping.

Django ORM allows developers to work with databases using Python objects and QuerySets instead of writing SQL for every operation.

Example:

```python
```

```
Student.objects.all()
```

Equivalent SQL concept:

```sql
SELECT * FROM students_student;
```

6.3 QuerySets and Lazy Evaluation

A QuerySet represents a collection of database records.

Django QuerySets are generally lazy: they do not execute the database query until the result is evaluated.

```python
students = Student.objects.filter(age__gte=18)
```

The query may execute when:

```python
list(students)
```

or:

```python
for student in students:
    print(student.name)
```

Other evaluation examples include `len()`, `bool()`, and `count()` under their respective circumstances.

6.4 all()

Returns all records.

```python
students = Student.objects.all()
```

6.5 get()

Returns exactly one object.

```python
student = Student.objects.get(id=1)
```

Possible exceptions:

- `DoesNotExist`
- `MultipleObjectsReturned`

6.6 filter()

Returns records matching conditions.

```python
students = Student.objects.filter(age=20)
```

6.7 exclude()

Returns records that do not match conditions.

```python
students = Student.objects.exclude(age=20)
```

get vs filter

get()	filter()
Returns one object	Returns QuerySet
Raises DoesNotExist	Empty QuerySet if no match
Raises MultipleObjectsReturned	Can return multiple records

6.8 create()

Creates and saves a new object.

```python
Student.objects.create(
    name="Faruk",
    age=24
)
```

6.9 save()

Creates or updates an object.

```python
student = Student(
    name="Faruk",
    age=24
)

student.save()
```

Update:

```python
student = Student.objects.get(id=1)

student.name = "Omar"

student.save()
```

6.10 update()

Updates matching records directly in the database.

```python
Student.objects.filter(id=1).update(
    name="Omar"
)
```

`update()` does not call each model instance's `save()` method or send normal `pre_save` and `post_save` signals.

6.11 delete()

```python
student = Student.objects.get(id=1)

student.delete()
```

Bulk delete:

```python
Student.objects.filter(age__lt=18).delete()
```

6.12 get_or_create()

Gets an existing record or creates one.

```python
student, created = Student.objects.get_or_create(
    name="Faruk",
    defaults={
        "age": 24
    }
)

print(created)
```

`created` is True when a new record was created.

6.13 update_or_create()

Updates an existing record or creates a new one.

```python
student, created = Student.objects.update_or_create(
    id=1,
    defaults={
        "name": "Faruk",
        "age": 25
    }
)
```

6.14 exists()

Checks whether matching records exist.

```python
Student.objects.filter(age=20).exists()
```

6.15 count()

Counts matching records.

```python
Student.objects.count()
```

6.16 first() and last()

```python
```

```
Student.objects.first()

Student.objects.last()
```

6.17 values()

Returns dictionaries containing selected fields.

```python
Student.objects.values("name", "age")
```

Example output:

```python
[
    {"name": "Faruk", "age": 24},
    {"name": "Omar", "age": 25}
]
```

6.18 values_list()

Returns tuples or a flat list.

```python
Student.objects.values_list("name", "age")
```

Flat list:

```python
Student.objects.values_list("name", flat=True)
```

6.19 only()

Loads selected fields initially.

```python
Student.objects.only("name")
```

Accessing deferred fields later may cause additional queries.

6.20 defer()

Defers loading selected fields.

```python
Student.objects.defer("description")
```

Useful when a field is large and not needed immediately.

6.21 order_by()

Ascending:

```python
Student.objects.order_by("name")
```

Descending:

```python
Student.objects.order_by("-age")
```

6.22 distinct()

Returns unique rows.

```python
Student.objects.values("age").distinct()
```

Database-specific behavior can apply when ordering and selecting distinct fields.

6.23 reverse()

Reverses the default or current ordering.

```python
Student.objects.order_by("name").reverse()
```

6.24 Query Lookups

```python
Student.objects.filter(age__gt=18)

Student.objects.filter(age__gte=18)

Student.objects.filter(age__lt=30)

Student.objects.filter(age__lte=30)

Student.objects.filter(name__exact="Faruk")

Student.objects.filter(name__iexact="faruk")

Student.objects.filter(name__contains="Far")

Student.objects.filter(name__icontains="far")

Student.objects.filter(name__startswith="F")

Student.objects.filter(name__endswith="k")

Student.objects.filter(age__in=[18, 20, 25])

Student.objects.filter(age__range=(18, 30))
```

6.25 Q Objects

Q objects create complex queries using AND, OR, and NOT.

```python
from django.db.models import Q

students = Student.objects.filter(
    Q(age=20) | Q(age=25)
)
```

AND:

```python
Student.objects.filter(
    Q(age__gte=18) & Q(is_active=True)
)
```

NOT:

```python
Student.objects.filter(
    ~Q(age=20)
)
```

6.26 F Expressions

F expressions reference database fields without first loading their values into Python.

```python
from django.db.models import F

Student.objects.update(
    age=F("age") + 1
)
```

Compare two fields:

```python
Student.objects.filter(
    age__gt=F("minimum_age")
)
```

Useful for database-side calculations and avoiding race conditions in many update operations.

6.27 Aggregation

Aggregation calculates a summary across a QuerySet.

```python
from django.db.models import (
    Count,
    Avg,
    Sum,
    Max,
    Min
)
```

Examples:

```python
Student.objects.aggregate(
    total=Count("id")
)

Student.objects.aggregate(
    average_age=Avg("age")
```

```
)

Student.objects.aggregate(
    maximum_age=Max("age")
)

Student.objects.aggregate(
    minimum_age=Min("age")
)

Student.objects.aggregate(
    total_age=Sum("age")
)
```

6.28 annotate()

`annotate()` adds calculated values to each returned object.

```python
from django.db.models import Count

departments = Department.objects.annotate(
    total_students=Count("students")
)

for department in departments:
    print(department.name, department.total_students)
```

annotate vs aggregate

annotate	aggregate
Adds value to each object/group	Returns summary dictionary
Works per returned row/group	Works over the QuerySet
Example: students per department	Example: total students

6.29 Conditional Expressions: Case and When

Conditional expressions provide SQL-like conditional logic.

```python
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    category=Case(
        When(age__gte=18, then=Value("Adult")),
        default=Value("Minor"),
        output_field=CharField()
    )
)
```

Each student gets a calculated category.

6.30 Subquery

A Subquery allows a query to be used inside another query.

```python
from django.db.models import OuterRef, Subquery

latest_task = Task.objects.filter(
```

```
    user=OuterRef("pk")
).order_by("-created_at")

users = User.objects.annotate(
    latest_task_name=Subquery(
        latest_task.values("title")[:1]
    )
)
```

6.31 OuterRef

OuterRef refers to a field from the outer query inside a subquery.

```python
OuterRef("pk")
```

It is commonly used with `Subquery` and `Exists`.

6.32 Exists

Checks whether a matching record exists inside a subquery.

```python
from django.db.models import Exists, OuterRef

active_tasks = Task.objects.filter(
    user=OuterRef("pk"),
    is_completed=False
)

users = User.objects.annotate(
    has_pending_tasks=Exists(active_tasks)
)
```

6.33 Relationships and Related-Object Queries

Forward ForeignKey access:

```python
student = Student.objects.get(id=1)

print(student.department.name)
```

Reverse ForeignKey access:

```python
department = Department.objects.get(id=1)

students = department.students.all()
```

ManyToMany access:

```python
student.courses.all()
```

Add relationship:

```python
```

```
student.courses.add(course)
```

Remove relationship:

```python
student.courses.remove(course)
```

Set relationships:

```python
student.courses.set([course1, course2])
```

Clear relationships:

```python
student.courses.clear()
```

6.34 select_related()

Used for ForeignKey and OneToOne relationships.

```python
students = Student.objects.select_related(
    "department"
)
```

It generally uses SQL JOIN to fetch related objects efficiently.

6.35 prefetch_related()

Used for ManyToMany and reverse relationships.

```python
students = Student.objects.prefetch_related(
    "courses"
)
```

It generally executes separate queries and combines the results in Python.

select_related vs prefetch_related

select_related	prefetch_related
SQL JOIN	Separate queries
ForeignKey	ManyToMany
OneToOneField	Reverse relationships
Single query in typical cases	Multiple queries in typical cases

6.36 Managers

A Manager is the interface through which Django models perform database queries.

Default manager:

```python
Student.objects.all()
```

Custom manager:

```python
class ActiveStudentManager(models.Manager):

    def get_queryset(self):
        return super().get_queryset().filter(
            is_active=True
        )

class Student(models.Model):

    name = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True)

    objects = models.Manager()
    active_students = ActiveStudentManager()
```

Usage:

```python
Student.active_students.all()
```

6.37 Custom QuerySets

Custom QuerySets allow reusable query methods.

```python
class StudentQuerySet(models.QuerySet):

    def active(self):
        return self.filter(is_active=True)

    def adults(self):
        return self.filter(age__gte=18)
```

Attach to model:

```python
class Student(models.Model):

    name = models.CharField(max_length=100)
    age = models.IntegerField()
    is_active = models.BooleanField(default=True)

    objects = StudentQuerySet.as_manager()
```

Usage:

```python
Student.objects.active()

Student.objects.adults()
```

6.38 Transactions: atomic()

A transaction groups multiple database operations into one atomic operation.

If one operation fails, the transaction can be rolled back.

```python
from django.db import transaction

with transaction.atomic():

    student = Student.objects.create(
        name="Faruk",
        age=24
    )

    Profile.objects.create(
        student=student
    )
```

Use transactions when multiple database changes must succeed or fail together.

6.39 Raw SQL and raw()

Django ORM is recommended for most queries.

Raw SQL can be used for specialized database operations.

```python
students = Student.objects.raw(
    "SELECT * FROM students_student"
)

for student in students:
    print(student.name)
```

Use parameterized queries:

```python
from django.db import connection

with connection.cursor() as cursor:

    cursor.execute(
        "SELECT * FROM students_student WHERE age = %s",
        [20]
    )

    rows = cursor.fetchall()
```

Never build SQL by directly concatenating untrusted input.

6.40 Bulk Create

Creates multiple objects efficiently.

```python
Student.objects.bulk_create([
    Student(name="A", age=20),
    Student(name="B", age=21),
    Student(name="C", age=22),
])
```

6.41 Bulk Update

```python
students = Student.objects.filter(
    age__gte=18
)

for student in students:
    student.is_active = True

Student.objects.bulk_update(
    students,
    ["is_active"]
)
```

6.42 Bulk Delete

```python
Student.objects.filter(
    age__lt=18
).delete()
```

Bulk operations can bypass normal per-instance `save()` behavior and signals. Check their behavior before relying on model hooks.

6.43 Database Indexes

An index helps the database find records more efficiently.

Field index:

```python
name = models.CharField(
    max_length=100,
    db_index=True
)
```

Model index:

```python
class Meta:
    indexes = [
        models.Index(fields=["name"]),
        models.Index(fields=["age", "name"]),
    ]
```

Indexes can improve reads but consume storage and add overhead to writes.

6.44 Query Optimization

Important optimization techniques:

1. Use `select_related()`.
2. Use `prefetch_related()`.
3. Use database indexes.
4. Avoid unnecessary queries.
5. Use pagination.
6. Use `only()` and `defer()` when appropriate.
7. Use bulk operations.
8. Inspect SQL queries.
9. Avoid N+1 query problems.
10. Profile before optimizing.

Inspect SQL:

```python
print(Student.objects.filter(age=20).query)
```

6.45 Multiple Databases

Django can connect to multiple databases.

Example:

```python
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    },

    "analytics": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "analytics.sqlite3",
    },
}
```

Use another database:

```python
Student.objects.using("analytics").all()
```

6.46 Database Routers

Database routers control which database is used for reads, writes, relations, and migrations.

Example:

```python
class AnalyticsRouter:

    route_app_labels = {"analytics"}

    def db_for_read(self, model, **hints):
        if model._meta.app_label in self.route_app_labels:
            return "analytics"
        return None

    def db_for_write(self, model, **hints):
        if model._meta.app_label in self.route_app_labels:
            return "analytics"
        return None
```

Register:

```python
DATABASE_ROUTERS = [
    "project.routers.AnalyticsRouter",
]
```

6.47 Fixtures

Fixtures are files containing initial or test data.

Export:

```bash
python manage.py dumpdata students
```

Load:

```bash
python manage.py loaddata students.json
```

Fixtures are useful for sample data, test data, and initial setup.

6.48 Data Migrations

Data migrations change existing database records using migration files.

Create an empty migration:

```bash
python manage.py makemigrations students --empty
```

Example:

```python
from django.db import migrations

def add_initial_data(apps, schema_editor):

    Student = apps.get_model("students", "Student")

    Student.objects.create(
        name="Faruk",
        age=24
    )

class Migration(migrations.Migration):

    dependencies = [
        ("students", "0001_initial"),
    ]

    operations = [
        migrations.RunPython(add_initial_data),
    ]
```

Use historical models through `apps.get_model()` in migrations instead of importing current model classes directly.

6.49 Signals

Signals allow applications to respond to certain events.

Common model signals:

- pre_save
- post_save

- pre_delete
- post_delete
- m2m_changed

Example:

```python
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=Student)
def student_created(sender, instance, created, **kwargs):

    if created:
        print("New student created:", instance.name)
```

Signals are useful for decoupled reactions, but explicit service functions are often easier to understand for important business logic.

6.50 Abstract Models

An abstract model provides reusable fields without creating a separate parent table.

```python
class BaseModel(models.Model):

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

class Student(BaseModel):
    name = models.CharField(max_length=100)
```

6.51 Proxy Models

A proxy model uses the same database table as the original model but changes Python-level behavior.

```python
class StudentProxy(Student):

    class Meta:
        proxy = True
        ordering = ["name"]
```

6.52 Multi-Table Inheritance

Multi-table inheritance creates a separate database table for each concrete model in the inheritance hierarchy.

```python
class Person(models.Model):
    name = models.CharField(max_length=100)

class Student(Person):
    roll = models.IntegerField()
```

The child model has its own table linked to the parent table.

6.53 Model Validation

Django models support validation through `full_clean()`.

```python
student = Student(
    name="Faruk",
    age=24
)

student.full_clean()
student.save()
```

Important validation methods:

- `clean_fields()`
- `clean()`
- `validate_unique()`
- `full_clean()`

Note: Calling `save()` does not automatically call `full_clean()`.

6.54 Database Constraints

Constraints enforce rules at the database level.

Unique constraint:

```python
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["email"],
            name="unique_student_email"
        )
    ]
```

Check constraint:

```python
class Meta:
    constraints = [
        models.CheckConstraint(
            condition=models.Q(age__gte=0),
            name="age_cannot_be_negative"
        )
    ]
```

6.55 Database Introspection

1. What is Database Introspection?

Database Introspection is the process of examining an existing database to discover its structure and metadata.

It helps us inspect:

Database tables.

Table columns.

Primary keys.

Foreign key relationships.

Column data types.

Existing database schemas.

Django provides database introspection tools through its database connection API.

2. Accessing Database Introspection

Django provides database introspection through:

```
from django.db import connection
```

The connection object provides access to the active database connection.

Its introspection attribute provides methods for examining database structures.

Example

```
from django.db import connection

print(connection.introspection)
```

3. Get All Database Table Names

We can use table_names() to retrieve the names of database tables.

Example

```
from django.db import connection

tables = connection.introspection.table_names()

print(tables)
```

Possible Output:

```
[
    'auth_group',
    'auth_user',
    'django_admin_log',
    'django_content_type',
    'django_migrations',
    'students'
]
```

The actual output depends on the database and installed Django applications.

Explanation

connection accesses the active database connection.

introspection provides database inspection functionality.

table_names() returns the available table names.

4. Understanding inspectdb

inspectdb is a Django management command that examines an existing database and generates Django model definitions based on its tables.

Command

```
python manage.py inspectdb
```

To inspect a specific table:

```
python manage.py inspectdb students
```

Example Generated Model

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    class Meta:
        managed = False
        db_table = "students"
```

Explanation

Django examines the existing database table.

It generates a model based on the discovered structure.

managed = False tells Django not to manage the table's creation or deletion through migrations.

db_table identifies the existing database table.

Important: inspectdb generates model code. You should review and adjust the generated models before using them in a production project.

5. Get Table Column Information

Django provides get_table_description() to retrieve information about the columns of a database table.

Example

```
from django.db import connection

with connection.cursor() as cursor:
    columns = connection.introspection.get_table_description(
        cursor,
        "students"
    )

    for column in columns:
        print(column.name)
```

Possible Output:

```
id
name
age
```

Explanation

connection.cursor() creates a database cursor.

get_table_description() retrieves column information.

column.name provides the name of each column.

The returned column description can contain additional metadata, depending on the database backend and Django version.

6. Get the Primary Key Column

We can use get_primary_key_column() to identify the primary key column of a database table.

Example

```
from django.db import connection

with connection.cursor() as cursor:
    pk_column = connection.introspection.get_primary_key_column(
        cursor,
        "students"
    )

    print(pk_column)
```

Possible Output:

```
id
```

Explanation

The method examines the specified table.

It identifies the primary key column.

The exact behavior and availability may depend on the Django version and database backend.

7. Get Foreign Key Relationships

Django provides get_relations() to inspect relationships between database columns.

Example

```
from django.db import connection

with connection.cursor() as cursor:
    relations = connection.introspection.get_relations(
        cursor,
        "students"
    )

    print(relations)
```

Explanation

This method retrieves relationship information for the specified table.

For example, a table might contain a foreign key referencing another table.

The returned structure depends on the database backend and Django version.

8. Important Database Introspection Methods

Method

Purpose

table_names()

Retrieves database table names

get_table_description()

Retrieves table column information

get_primary_key_column()

Identifies the primary key column

get_relations()

Retrieves database relationship information

inspectdb

Generates Django models from existing database tables

9. Model Introspection vs Database Introspection

Model Introspection

Database Introspection

Examines Django model metadata

Examines the database structure

Uses the _meta API

Uses connection.introspection

Works with Django model definitions

Works with actual database tables

Example: Student._meta.get_fields()

Example: table_names()

Useful for dynamic model operations

Useful for inspecting existing database schemas

Model Introspection Example

```
for field in Student._meta.get_fields():
    print(field.name)
```

Database Introspection Example

```
from django.db import connection

print(connection.introspection.table_names())
```

10. Practical Use Cases

Database introspection is useful for:

Working with legacy databases.

Connecting Django to an existing database.

Generating models from existing tables.

Inspecting database schemas.

Database migration analysis.

Building database inspection tools.

Understanding existing foreign key relationships.

11. Interview Questions and Answers

Q1. What is database introspection?

Database introspection is the process of examining an existing database to discover its tables, columns, primary keys, and relationships.

Q2. How can you access database introspection in Django?

```
from django.db import connection

connection.introspection
```

Q3. How can you get all database table names?

```
connection.introspection.table_names()
```

Q4. What is inspectdb?

inspectdb is a Django management command that generates Django model definitions from existing database tables.

Q5. What does managed = False mean?

It tells Django not to manage the database table's creation or deletion through migrations.

Q6. How can you inspect the columns of a database table?

Use get_table_description() with a database cursor.

Q7. What is the difference between model introspection and database introspection?

Model introspection examines Django model metadata, while database introspection examines the actual database schema.

12. Practice Tasks

Import connection from django.db.

Print all database table names.

Inspect the columns of the students table.

Identify the primary key column.

Inspect the foreign key relationships of a table.

Run the inspectdb command.

Generate Django models from an existing database table.

Explain the purpose of managed = False.

Quick Revision

Topic

Main Concept

Introspection

Examining objects and structures at runtime

__class__

Returns an object's class

Meta

Defines Django model configuration

_meta

Provides Django model metadata

get_fields()

Retrieves model field information

connection

Provides access to the active database connection

connection.introspection

Provides database inspection methods

table_names()

Retrieves database table names

inspectdb

Generates models from existing database tables

get_table_description()

Retrieves table column information

get_primary_key_column()

Identifies the primary key column

get_relations()

Inspects database relationships

Key Difference to Remember:

Student._meta → Examines Django model metadata.

connection.introspection → Examines the actual database structure.

6.56 ORM Security and SQL Injection

SQL injection occurs when untrusted input is inserted into SQL in an unsafe way.

Unsafe pattern:

```python
# Do not build SQL using string concatenation.
```

Safe ORM:

```python
Student.objects.filter(name=user_input)
```

Safe raw SQL:

```python
cursor.execute(
    "SELECT * FROM students_student WHERE name = %s",
    [user_input]
)
```

Always use ORM lookups or parameterized SQL.

6.57 ORM Cheat Sheet

Task	ORM
Get all	`Model.objects.all()`
Get one	`Model.objects.get()`
Filter	`Model.objects.filter()`
Exclude	`Model.objects.exclude()`
Create	`Model.objects.create()`
Update instance	`obj.save()`
Bulk update	`QuerySet.update()`
Delete	`obj.delete()`
Count	`QuerySet.count()`
Exists	`QuerySet.exists()`
First	`QuerySet.first()`
Last	`QuerySet.last()`
Order	`QuerySet.order_by()`
Unique	`QuerySet.distinct()`
Values	`QuerySet.values()`
Aggregate	`QuerySet.aggregate()`
Annotate	`QuerySet.annotate()`
Join FK	`select_related()`
Prefetch relations	`prefetch_related()`

6.58 ORM Interview Questions

1. What is ORM?
2. What is a QuerySet?
3. What is lazy evaluation?
4. Difference between `get()` and `filter()`?
5. Difference between `save()` and `update()`?
6. What is `get_or_create()`?
7. What is `update_or_create()`?
8. What are Q objects?
9. What are F expressions?
10. What is aggregation?

11. Difference between `annotate()` and `aggregate()`?
12. What is Subquery?
13. What is OuterRef?
14. What is Exists?
15. Difference between `select_related()` and `prefetch_related()`?
16. What is a custom manager?
17. What is a custom QuerySet?
18. What is `transaction.atomic()`?
19. What is a database index?
20. What is the N+1 query problem?
21. What is a data migration?
22. What are Django signals?
23. What is a proxy model?
24. What is an abstract model?
25. How do you prevent SQL injection?

---

7. Django Forms

7.1 What is a Form?

A Django Form handles user input, validation, and cleaned data.

7.2 Form

```python
from django import forms

class StudentForm(forms.Form):

    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    age = forms.IntegerField()
```

7.3 ModelForm

ModelForm creates form fields based on a Django model.

```python
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = ["name", "age"]
```

7.4 Form Fields

Common fields:

- CharField
- EmailField
- IntegerField
- DecimalField
- BooleanField
- DateField
- DateTimeField
- ChoiceField
- MultipleChoiceField

- FileField
- ImageField
- URLField

7.5 Widgets

Widgets control how form fields are displayed in HTML.

```python
class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = ["name", "age"]

        widgets = {
            "name": forms.TextInput(
                attrs={"class": "form-control"}
            ),

            "age": forms.NumberInput(
                attrs={"class": "form-control"}
            ),
        }
```

Common widgets:

- TextInput
- PasswordInput
- Textarea
- NumberInput
- EmailInput
- Select
- CheckboxInput
- DateInput
- FileInput

7.6 Form Validation

```python
class StudentForm(forms.ModelForm):

    class Meta:
        model = Student
        fields = ["name", "age"]

    def clean_age(self):

        age = self.cleaned_data["age"]

        if age < 18:
            raise forms.ValidationError(
                "Age must be at least 18"
            )

        return age
```

7.7 clean() Method

Used for validation involving multiple fields.

```python
```

```
def clean(self):

    cleaned_data = super().clean()

    name = cleaned_data.get("name")
    age = cleaned_data.get("age")

    if name and age and age < 18:
        raise forms.ValidationError(
            "Student must be an adult"
        )

    return cleaned_data
```

7.8 cleaned_data

After successful validation, cleaned data contains validated values.

```python
if form.is_valid():

    name = form.cleaned_data["name"]
    age = form.cleaned_data["age"]
```

7.9 errors

```python
if not form.is_valid():
    print(form.errors)
```

7.10 save() and commit=False

Save ModelForm:

```python
if form.is_valid():
    form.save()
```

Save later:

```python
if form.is_valid():

    student = form.save(commit=False)

    student.name = student.name.upper()

    student.save()
```

For many-to-many relationships, call `form.save_m2m()` after saving the instance when required.

7.11 File Uploads

Form:

```python
class DocumentForm(forms.ModelForm):
```

```
    class Meta:
        model = Document
        fields = ["title", "file"]
```

Template:

```html
<form method="POST" enctype="multipart/form-data">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Upload</button>

</form>
```

View:

```python
def upload_document(request):

    if request.method == "POST":

        form = DocumentForm(
            request.POST,
            request.FILES
        )

        if form.is_valid():
            form.save()
            return redirect("document_list")

    else:
        form = DocumentForm()

    return render(
        request,
        "upload.html",
        {"form": form}
    )
```

7.12 Display Validation Errors

```html
<form method="POST">
    {% csrf_token %}

    {{ form.non_field_errors }}

    {{ form.name.errors }}
    {{ form.name }}

    {{ form.age.errors }}
    {{ form.age }}

    <button type="submit">Save</button>
</form>
```

---

8. Authentication and Authorization

8.1 Authentication

Authentication verifies who the user is.

Example: Login using username and password.

8.2 Authorization

Authorization determines what an authenticated user can access.

Example: A normal user cannot access an admin-only feature.

8.3 User Model

Django provides a built-in User model.

```python
from django.contrib.auth.models import User
```

8.4 Create User

```python
user = User.objects.create_user(
    username="faruk",
    email="faruk@example.com",
    password="strong-password"
)
```

8.5 Create Superuser

```bash
python manage.py createsuperuser
```

8.6 Login

```python
from django.contrib.auth import authenticate, login

def user_login(request):

    username = request.POST.get("username")
    password = request.POST.get("password")

    user = authenticate(
        request,
        username=username,
        password=password
    )

    if user is not None:
        login(request, user)
        return redirect("dashboard")

    return HttpResponse("Invalid credentials")
```

8.7 Logout

```python
from django.contrib.auth import logout

def user_logout(request):

    logout(request)

    return redirect("login")
```

8.8 Password Hashing

Django does not store passwords as plain text.

It uses password hashers to store password hashes.

Create password:

```python
user.set_password("new-password")
user.save()
```

Check password:

```python
user.check_password("new-password")
```

8.9 Password Reset

Django provides built-in password reset views.

Common views:

- PasswordResetView
- PasswordResetDoneView
- PasswordResetConfirmView
- PasswordResetCompleteView

These can be configured with URL patterns and email settings.

8.10 Sessions

Sessions store user-specific data on the server side, associated with a session identifier.

Set:

```python
request.session["username"] = "Faruk"
```

Get:

```python
username = request.session.get("username")
```

Delete:

```python
del request.session["username"]
```

Clear:

```python
request.session.flush()
```

8.11 Cookies

Cookies are small pieces of data stored in the browser.

Set:

```python
response = HttpResponse("Cookie Set")

response.set_cookie(
    "username",
    "Faruk",
    httponly=True,
    secure=True,
    samesite="Lax"
)

return response
```

Get:

```python
username = request.COOKIES.get("username")
```

Delete:

```python
response.delete_cookie("username")
```

8.12 Permissions

Django provides model permissions:

- add
- change
- delete
- view

Check permission:

```python
request.user.has_perm(
    "students.view_student"
)
```

8.13 Groups

Groups allow permissions to be assigned to multiple users.

```python
from django.contrib.auth.models import Group

group = Group.objects.create(
```

````
    name="Managers"
)
```

Add user:

```python
user.groups.add(group)
```

## 8.14 @login_required

Protects views so that only authenticated users can access them.

```python
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return HttpResponse("Dashboard")
```

## 8.15 Custom User Model

A custom user model allows additional user fields or customized authentication behavior.

```python
from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):

    phone = models.CharField(
        max_length=20,
        blank=True
    )
```

Settings:

```python
AUTH_USER_MODEL = "users.CustomUser"
```

Define the custom user model before the first migration whenever possible.

## 8.16 Authentication Backends

Authentication backends determine how Django authenticates users.

Example:

```python
AUTHENTICATION_BACKENDS = [
    "django.contrib.auth.backends.ModelBackend",
]
```

Custom backends can support alternative login methods such as email or external authentication.

## 8.17 JWT Authentication Concept
````
````

JWT = JSON Web Token.

A JWT commonly contains:

- Header
- Payload
- Signature

Typical authentication flow:

1. User submits credentials.
2. Server validates credentials.
3. Server returns an access token and possibly a refresh token.
4. Client sends the access token with API requests.
5. Server verifies the token.
6. Server checks permissions.

JWT is usually stateless from the server's perspective, but token expiry, revocation, refresh, and storage must be designed carefully.

8.18 Token Authentication

Token authentication uses a token to identify an authenticated API client.

The client sends:

```http
Authorization: Token your-token
```

JWT commonly uses:

```http
Authorization: Bearer your-access-token
```

---

9. Django REST Framework (DRF)

9.1 What is DRF?

Django REST Framework is a toolkit for building REST APIs using Django.

It provides:

- Serializers
- API Views
- Authentication
- Permissions
- Pagination
- Filtering
- Throttling
- Browsable API
- API testing tools

9.2 Install DRF

```bash
pip install djangorestframework
```

Register:

```python
INSTALLED_APPS = [
    "rest_framework",
]
```

9.3 Request and Response

DRF Request provides parsed request data.

```python
request.data
```

DRF Response returns API content.

```python
from rest_framework.response import Response

return Response({
    "message": "Success"
})
```

9.4 Serializer

A serializer converts model instances into Python data that can be rendered as JSON and validates incoming data.

```python
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = "__all__"
```

9.5 ModelSerializer

ModelSerializer automatically generates serializer fields based on a model.

```python
class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = ["id", "name", "age"]
```

9.6 Serializer Validation

Field validation:

```python
class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = "__all__"

    def validate_age(self, value):
```

```
        if value < 18:
            raise serializers.ValidationError(
                "Age must be at least 18"
            )

        return value
```

Object-level validation:

```python
def validate(self, attrs):

    if attrs["age"] < 18:
        raise serializers.ValidationError(
            "Student must be an adult"
        )

    return attrs
```

9.7 @api_view

Function-based API view:

```python
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(["GET"])
def hello_api(request):

    return Response({
        "message": "Hello API"
    })
```

9.8 APIView

APIView uses a Python class to handle HTTP methods.

```python
from rest_framework.views import APIView
from rest_framework.response import Response

class StudentAPIView(APIView):

    def get(self, request):
        return Response({
            "message": "GET Request"
        })

    def post(self, request):
        return Response({
            "message": "POST Request"
        })
```

9.9 Generic Views

Generic views provide reusable implementations for common API operations.

Examples:

- ListAPIView
- CreateAPIView
- RetrieveAPIView
- UpdateAPIView
- DestroyAPIView
- ListCreateAPIView
- RetrieveUpdateDestroyAPIView

Example:

```python
from rest_framework import generics

class StudentListCreateAPIView(
    generics.ListCreateAPIView
):

    queryset = Student.objects.all()

    serializer_class = StudentSerializer
```

9.10 Mixins

Mixins provide reusable API actions.

- ListModelMixin
- CreateModelMixin
- RetrieveModelMixin
- UpdateModelMixin
- DestroyModelMixin

Example:

```python
from rest_framework import generics, mixins

class StudentListCreateAPIView(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    generics.GenericAPIView
):

    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)
```

9.11 ViewSets

ViewSets group related API actions into one class.

```python
from rest_framework import viewsets

class StudentViewSet(viewsets.ModelViewSet):

    queryset = Student.objects.all()
```

```
    serializer_class = StudentSerializer
```

ModelViewSet provides standard CRUD actions.

9.12 Routers

Routers automatically generate URL patterns for ViewSets.

```python
from rest_framework.routers import DefaultRouter

router = DefaultRouter()

router.register(
    "students",
    StudentViewSet,
    basename="student"
)

urlpatterns = router.urls
```

9.13 HTTP Status Codes

Code	Meaning
200	OK
201	Created
202	Accepted
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict
429	Too Many Requests
500	Internal Server Error

9.14 CRUD APIs

Create

```python
@api_view(["POST"])
def student_create(request):

    serializer = StudentSerializer(
        data=request.data
    )

    if serializer.is_valid():

        serializer.save()

        return Response(
            serializer.data,
            status=201
        )

    return Response(
        serializer.errors,
```

```
        status=400
    )
```

Read

```python
@api_view(["GET"])
def student_list(request):

    students = Student.objects.all()

    serializer = StudentSerializer(
        students,
        many=True
    )

    return Response(serializer.data)
```

Update

```python
@api_view(["PUT", "PATCH"])
def student_update(request, pk):

    student = get_object_or_404(Student, pk=pk)

    serializer = StudentSerializer(
        student,
        data=request.data,
        partial=request.method == "PATCH"
    )

    if serializer.is_valid():

        serializer.save()

        return Response(serializer.data)

    return Response(serializer.errors, status=400)
```

Delete

```python
@api_view(["DELETE"])
def student_delete(request, pk):

    student = get_object_or_404(Student, pk=pk)

    student.delete()

    return Response(status=204)
```

9.15 Pagination

Pagination divides large API responses into smaller pages.

Settings:

```python
REST_FRAMEWORK = {
```

```
    "DEFAULT_PAGINATION_CLASS":
        "rest_framework.pagination.PageNumberPagination",

    "PAGE_SIZE": 10,
}
```

Common pagination styles:

- PageNumberPagination
- LimitOffsetPagination
- CursorPagination

9.16 Filtering

Install:

```bash
pip install django-filter
```

Register:

```python
INSTALLED_APPS = [
    "django_filters",
]
```

ViewSet:

```python
class StudentViewSet(viewsets.ModelViewSet):

    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    filterset_fields = ["age"]
```

Example URL:

```text
/api/students/?age=20
```

9.17 Searching

```python
from rest_framework.filters import SearchFilter

class StudentViewSet(viewsets.ModelViewSet):

    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    filter_backends = [SearchFilter]

    search_fields = ["name", "email"]
```

Example:

```text
```

```
/api/students/?search=Faruk
```

9.18 Ordering

```python
from rest_framework.filters import OrderingFilter

class StudentViewSet(viewsets.ModelViewSet):

    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    filter_backends = [OrderingFilter]

    ordering_fields = ["name", "age"]
```

Example:

```text
/api/students/?ordering=-age
```

9.19 Authentication and Permissions

Authentication identifies the requester.

Permissions determine whether the requester can perform an action.

```python
from rest_framework.permissions import IsAuthenticated

class StudentViewSet(viewsets.ModelViewSet):

    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    permission_classes = [IsAuthenticated]
```

Common permissions:

- AllowAny
- IsAuthenticated
- IsAdminUser
- IsAuthenticatedOrReadOnly
- DjangoModelPermissions

9.20 Token Authentication

Install and configure DRF TokenAuthentication if you want built-in token authentication.

```python
INSTALLED_APPS = [
    "rest_framework",
    "rest_framework.authtoken",
]
```

Run migrations:

```bash
```

````
python manage.py migrate
```

Settings:

```python
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
    ],
}
```

Header:

```http
Authorization: Token your-token
```

## 9.21 JWT Authentication

Install:

```bash
pip install djangorestframework-simplejwt
```

Settings:

```python
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ],
}
```

URLs:

```python
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView
)

urlpatterns = [
    path("token/", TokenObtainPairView.as_view()),
    path("token/refresh/", TokenRefreshView.as_view()),
]
```

Obtain token:

```http
POST /token/
```

Example response:

```json
{
    "refresh": "refresh-token",
    "access": "access-token"
}
````

```

Send access token:

```http
Authorization: Bearer access-token
```

9.22 Throttling and Rate Limits

Throttling limits how frequently a client can access an API.

```python
REST_FRAMEWORK = {
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],

    "DEFAULT_THROTTLE_RATES": {
        "anon": "100/day",
        "user": "1000/day",
    },
}
```

9.23 API Versioning

API versioning allows an API to evolve while maintaining compatibility.

Common strategies:

- URLPathVersioning
- NamespaceVersioning
- AcceptHeaderVersioning
- QueryParameterVersioning

Example:

```text
/api/v1/students/
/api/v2/students/
```

9.24 File Upload APIs

Serializer:

```python
class DocumentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Document
        fields = "__all__"
```

View:

```python
from rest_framework.parsers import MultiPartParser, FormParser

class DocumentUploadAPIView(APIView):

    parser_classes = [

```
        MultiPartParser,
        FormParser
    ]

    def post(self, request):

        serializer = DocumentSerializer(
            data=request.data
        )

        if serializer.is_valid():

            serializer.save()

            return Response(
                serializer.data,
                status=201
            )

        return Response(
            serializer.errors,
            status=400
        )
```

9.25 Nested Serializers

Nested serializers represent related objects inside another serializer.

```python
class CourseSerializer(serializers.ModelSerializer):

    class Meta:
        model = Course
        fields = ["id", "name"]

class StudentSerializer(serializers.ModelSerializer):

    courses = CourseSerializer(
        many=True,
        read_only=True
    )

    class Meta:
        model = Student
        fields = ["id", "name", "courses"]
```

Writable nested serializers need explicit create/update handling or another appropriate relationship strategy.

9.26 Relationship Fields

Common relationship serializer fields:

- PrimaryKeyRelatedField
- StringRelatedField
- SlugRelatedField
- HyperlinkedRelatedField
- HyperlinkedIdentityField

Example:

```python
```

```python
class StudentSerializer(serializers.ModelSerializer):

    department = serializers.PrimaryKeyRelatedField(
        queryset=Department.objects.all()
    )

    class Meta:
        model = Student
        fields = "__all__"
```

9.27 Exception Handling

DRF provides API exception classes.

```python
from rest_framework.exceptions import (
    ValidationError,
    NotFound,
    PermissionDenied
)
```

Example:

```python
if not request.user.is_authenticated:
    raise PermissionDenied("Login required")
```

Custom exception handler:

```python
REST_FRAMEWORK = {
    "EXCEPTION_HANDLER":
        "myapp.exceptions.custom_exception_handler",
}
```

9.28 API Testing

```python
from rest_framework.test import APITestCase
from rest_framework import status

class StudentAPITest(APITestCase):

    def test_student_list(self):

        response = self.client.get("/api/students/")

        self.assertEqual(
            response.status_code,
            status.HTTP_200_OK
        )
```

9.29 OpenAPI and Swagger Documentation

OpenAPI is a specification for describing APIs.

Swagger UI provides an interactive API documentation interface.

A commonly used DRF package is:

```bash
pip install drf-spectacular
```

Settings:

```python
INSTALLED_APPS = [
    "drf_spectacular",
]
```

Configure:

```python
REST_FRAMEWORK = {
    "DEFAULT_SCHEMA_CLASS":
        "drf_spectacular.openapi.AutoSchema",
}
```

URLs:

```python
from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularSwaggerView
)

urlpatterns = [
    path("schema/", SpectacularAPIView.as_view(), name="schema"),

    path(
        "docs/",
        SpectacularSwaggerView.as_view(url_name="schema"),
        name="swagger-ui"
    ),
]
```

---

10. Testing and Debugging

10.1 What is Testing?

Testing verifies that application code works as expected.

Types:

- Unit Testing
- Integration Testing
- Functional Testing
- API Testing
- Regression Testing

10.2 Django TestCase

```python
from django.test import TestCase
from .models import Student

class StudentTestCase(TestCase):
```

```
    def setUp(self):

        Student.objects.create(
            name="Faruk",
            age=24
        )

    def test_student_exists(self):

        student = Student.objects.get(
            name="Faruk"
        )

        self.assertEqual(student.age, 24)
```

10.3 Unit Tests

Unit tests verify a small piece of functionality.

```python
def add(a, b):
    return a + b
```

Test:

```python
def test_add():
    assert add(2, 3) == 5
```

10.4 Integration Tests

Integration tests verify that multiple components work together.

Example:

- View
- Serializer
- Model
- Database

10.5 Test Client

Django Test Client simulates HTTP requests.

```python
from django.test import TestCase

class HomeTest(TestCase):

    def test_home(self):

        response = self.client.get("/")

        self.assertEqual(response.status_code, 200)
```

10.6 Testing Models

```python
def test_student_name(self):
```

```
    student = Student.objects.create(
        name="Faruk",
        age=24
    )

    self.assertEqual(student.name, "Faruk")
```

10.7 Testing Views

```python
def test_student_list(self):

    response = self.client.get("/students/")

    self.assertEqual(response.status_code, 200)
```

10.8 Testing Forms

```python
def test_valid_form(self):

    form = StudentForm(
        data={
            "name": "Faruk",
            "age": 24
        }
    )

    self.assertTrue(form.is_valid())
```

10.9 Testing APIs

```python
from rest_framework.test import APITestCase

class StudentAPITest(APITestCase):

    def test_api(self):

        response = self.client.get("/api/students/")

        self.assertEqual(response.status_code, 200)
```

10.10 Mocking External Services

Mocking replaces an external dependency with a controlled fake during tests.

Example:

```python
from unittest.mock import patch

@patch("myapp.services.send_email")
def test_email(mock_send_email):

    mock_send_email.return_value = True

    result = mock_send_email()
```

```
    assert result is True
```

10.11 Debugging Errors and Tracebacks

A traceback shows where an error occurred.

Common errors:

- ImportError
- ModuleNotFoundError
- TypeError
- ValueError
- AttributeError
- DoesNotExist
- IntegrityError
- ValidationError
- OperationalError

Debugging steps:

1. Read the full traceback.
2. Find the last relevant application line.
3. Check variable values.
4. Check imports and dependencies.
5. Check database migrations.
6. Reproduce the error.
7. Write a regression test after fixing it.

10.12 Logging

Logging records application events and errors.

```python
import logging

logger = logging.getLogger(__name__)

def my_view(request):

    logger.info("View called")

    try:
        result = 10 / 2

    except Exception:
        logger.exception("Unexpected error")
        raise
```

Common logging levels:

- DEBUG
- INFO
- WARNING
- ERROR
- CRITICAL

10.13 Test Database and Settings

Django creates a separate test database for tests.

Run:

```bash
python manage.py test
```

Use separate test settings when your project needs different testing configuration.

Avoid running destructive tests against production databases.

---

11. Security

11.1 CSRF Protection

CSRF = Cross-Site Request Forgery.

Use Django's CSRF middleware and template token for standard form submissions.

```html
<form method="POST">
    {% csrf_token %}
</form>
```

11.2 XSS Prevention

XSS = Cross-Site Scripting.

Django templates escape many variable values by default.

Avoid rendering untrusted HTML as safe.

11.3 SQL Injection Prevention

Use ORM:

```python
Student.objects.filter(name=user_input)
```

Use parameterized SQL when raw SQL is necessary.

Never concatenate untrusted values into SQL strings.

11.4 Clickjacking Protection

Clickjacking tricks a user into interacting with a hidden or misleading frame.

Django provides clickjacking protection through middleware and frame-related response headers.

11.5 Secure Password Storage

Use Django password hashing.

```python
user.set_password("strong-password")
```

Never store plain-text passwords.

11.6 HTTPS and Secure Cookies

Use HTTPS in production.

Important settings:

```python
SESSION_COOKIE_SECURE = True

CSRF_COOKIE_SECURE = True

SECURE_SSL_REDIRECT = True
```

Configure HTTPS at the reverse proxy and application level correctly.

11.7 Secret Keys and Environment Variables

Never expose:

- SECRET_KEY
- Database passwords
- API keys
- JWT signing secrets
- Email passwords

Use environment variables or a secrets manager.

11.8 DEBUG=False

Production:

```python
DEBUG = False
```

Debug mode can expose sensitive error information.

11.9 Allowed Hosts

```python
ALLOWED_HOSTS = [
    "example.com",
    "www.example.com",
]
```

Configure the actual production domains.

11.10 CORS Configuration

CORS = Cross-Origin Resource Sharing.

CORS controls which browser origins may access cross-origin resources.

Install:

```bash
pip install django-cors-headers
```

Register:

```python
INSTALLED_APPS = [
    "corsheaders",
```

```
]
```

Add middleware near the top:

```python
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
]
```

Configure allowed origins:

```python
CORS_ALLOWED_ORIGINS = [
    "https://frontend.example.com",
]
```

CORS is not authentication or authorization. It is a browser cross-origin policy.

11.11 Authentication and Permission Checks

Always verify:

- Who is making the request?
- Is the user authenticated?
- Does the user have permission?
- Does the requested object belong to that user?

Never rely only on frontend restrictions.

11.12 File Upload Validation

Validate:

- File size
- File extension
- Actual content type
- File content
- User permissions

Do not trust a filename or client-provided MIME type alone.

Store uploaded files safely and avoid executing uploaded content.

11.13 Dependency Updates

Regularly update dependencies.

```bash
pip list --outdated
```

Check dependency vulnerabilities using appropriate security tools.

---

12. Performance and Optimization

12.1 Query Optimization

Optimize database queries to reduce execution time and database load.

Use:

- `select_related()`
- `prefetch_related()`
- `only()`
- `defer()`
- `values()`
- `values_list()`
- Indexes
- Pagination
- Bulk operations

12.2 N+1 Query Problem

N+1 happens when one query loads a collection and additional queries are executed for each related object.

Example problem:

```python
students = Student.objects.all()

for student in students:
    print(student.department.name)
```

If there are many students, this can cause many additional database queries.

Solution:

```python
students = Student.objects.select_related(
    "department"
)
```

12.3 Database Indexes

Indexes speed up certain database lookups.

```python
class Student(models.Model):

    email = models.EmailField(db_index=True)
```

Use indexes based on real query patterns.

12.4 Caching

Caching stores frequently used data so it does not need to be recalculated or fetched repeatedly.

```python
from django.core.cache import cache

cache.set(
    "username",
    "Faruk",
    timeout=300
)

username = cache.get("username")
```

```

Delete:

```python
cache.delete("username")
```

Common cache backends:

- Local memory cache
- File-based cache
- Database cache
- Redis
- Memcached

12.5 Pagination

Pagination reduces the amount of data returned in one request.

```python
REST_FRAMEWORK = {
    "DEFAULT_PAGINATION_CLASS":
        "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 20,
}
```

12.6 Static and Media Files

Static files:

- CSS
- JavaScript
- Fonts
- Design images

Media files:

- User-uploaded images
- Documents
- Videos

Production static and media files are commonly served through a web server or object storage.

12.7 Profiling and Query Inspection

Inspect QuerySet SQL:

```python
print(
    Student.objects.filter(age=20).query
)
```

Django Debug Toolbar can help inspect queries during development.

Profiling helps identify actual performance bottlenecks before optimizing.

12.8 Bulk Database Operations

Bulk create:
```

```python
Student.objects.bulk_create([
    Student(name="A", age=20),
    Student(name="B", age=21),
])
```

Bulk update:

```python
students = list(Student.objects.all())

for student in students:
    student.is_active = True

Student.objects.bulk_update(
    students,
    ["is_active"]
)
```

12.9 Background Jobs and Task Queues

Background jobs execute long-running work outside the normal request-response cycle.

Examples:

- Sending emails
- Generating reports
- Processing images
- Scheduled notifications
- Large data imports

Popular tools:

- Celery
- Redis
- Django-Q alternatives
- RQ

Example Celery task:

```python
from celery import shared_task

@shared_task
def send_notification():

    print("Notification sent")
```

A task queue requires appropriate broker and worker configuration.

---

13. Reusable Django Apps and APIs

13.1 What is a Reusable App?

A reusable Django app is designed to be installed and used in multiple Django projects.

Examples:

- Authentication app
- Notification app
- Payment app
- Common utilities app

13.2 Create Reusable App

```bash
python manage.py startapp common
```

Example structure:

```text
common/
│
├── common/
│   ├── __init__.py
│   ├── apps.py
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
│
├── tests/
├── README.md
└── pyproject.toml
```

13.3 Separate Business Logic from Views

Keep views focused on HTTP request and response handling.

Business logic can live in service functions.

Example `services.py`:

```python
from django.db import transaction

@transaction.atomic
def create_student(name, age):

    student = Student.objects.create(
        name=name,
        age=age
    )

    return student
```

View:

```python
def student_create(request):

    student = create_student(
        name=request.POST["name"],
        age=int(request.POST["age"])
    )

    return JsonResponse({
        "id": student.id,
```

```
        "name": student.name
    })
```

In real projects, validate input before passing it to the service.

13.4 Reusable API Response Helpers

A response helper can keep API response formats consistent.

```python
from rest_framework.response import Response

def success_response(
    data=None,
    message="Success",
    status_code=200
):

    return Response(
        {
            "success": True,
            "message": message,
            "data": data
        },
        status=status_code
    )
```

Error helper:

```python
def error_response(
    message="Something went wrong",
    status_code=400
):

    return Response(
        {
            "success": False,
            "message": message,
            "data": None
        },
        status=status_code
    )
```

13.5 Error Response Conventions

Consistent API error response:

```json
{
    "success": false,
    "message": "Validation failed",
    "errors": {
        "email": [
            "Enter a valid email address."
        ]
    }
}
```

Use suitable HTTP status codes.

13.6 Input Validation

Validate incoming data at the API boundary.

```python
class StudentSerializer(serializers.ModelSerializer):

    class Meta:
        model = Student
        fields = ["name", "age"]

    def validate_age(self, value):

        if value < 18:
            raise serializers.ValidationError(
                "Age must be at least 18"
            )

        return value
```

13.7 API Tests

Test:

- Successful requests
- Invalid requests
- Authentication
- Permissions
- Validation
- Database behavior
- Error responses

13.8 Package Configuration

Use `pyproject.toml` to define package metadata and build configuration.

A reusable app should include:

- Package metadata
- Dependencies
- Tests
- Documentation
- License
- Version
- Installation instructions

13.9 Documentation and Versioning

Document:

- Installation
- Configuration
- Usage
- API endpoints
- Examples
- Compatibility
- Version history

Use semantic versioning:

```text
MAJOR.MINOR.PATCH
```

```

Example:

```text
1.2.3
```

---

14. Deployment

14.1 What is Deployment?

Deployment means making a Django application available in a production environment.

14.2 Production Settings

Important settings:

```python
DEBUG = False

ALLOWED_HOSTS = [
    "example.com"
]

SECRET_KEY = os.getenv("SECRET_KEY")
```

Use environment-specific configuration.

14.3 Environment Variables

Example `.env`:

```env
SECRET_KEY=your-production-secret
DEBUG=False
DATABASE_NAME=mydatabase
DATABASE_USER=myuser
DATABASE_PASSWORD=your-password
```

Never commit production `.env` files.

14.4 WSGI Deployment

WSGI is commonly used for traditional synchronous Django deployment.

Example:

```bash
gunicorn config.wsgi:application
```

Install:

```bash
pip install gunicorn
```

14.5 ASGI Deployment
```

ASGI supports asynchronous Django features.

Example:

```bash
uvicorn config.asgi:application
```

Install:

```bash
pip install uvicorn
```

14.6 Static Files and collectstatic

Settings:

```python
STATIC_URL = "static/"

STATIC_ROOT = BASE_DIR / "staticfiles"
```

Run:

```bash
python manage.py collectstatic
```

14.7 Media Storage

Uploaded files should be stored in a suitable persistent storage location.

Possible options:

- Server storage
- Object storage
- Cloud storage

Do not assume local ephemeral server storage is permanent.

14.8 Database Migrations During Deployment

Run migrations carefully:

```bash
python manage.py migrate
```

Before deploying:

1. Back up the database.
2. Review migrations.
3. Apply migrations.
4. Verify application behavior.

For complex schema changes, use backward-compatible migration strategies.

14.9 Gunicorn and Uvicorn

Gunicorn: Common WSGI server for Django.

Uvicorn: ASGI server commonly used for asynchronous Python applications.

14.10 Reverse Proxy

A reverse proxy receives incoming requests and forwards them to the application server.

Popular reverse proxies:

- Nginx
- Apache

It can handle:

- HTTPS termination
- Static files
- Request forwarding
- Load balancing
- Compression

14.11 HTTPS and Domain Configuration

Production applications should use HTTPS.

Configure:

- Domain DNS
- SSL/TLS certificate
- Reverse proxy
- Secure cookies
- HTTPS redirects
- Trusted proxy settings where needed

14.12 Logging and Monitoring

Monitor:

- Application errors
- Request failures
- Database performance
- Server resources
- Background job failures
- Security events

14.13 Backups and Rollback Strategy

Maintain:

- Database backups
- Media backups
- Deployment version history
- Migration strategy
- Rollback procedure

A backup is useful only if restoration has been tested.

---

15. Python Packaging

15.1 What is Python Packaging?

Python packaging means organizing code into distributable packages that can be installed and reused in other projects.

15.2 pyproject.toml

`pyproject.toml` is the modern standard configuration file for Python project metadata and build tools.

Example:

```toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "my-django-utils"
version = "0.1.0"
description = "Reusable Django utilities"
readme = "README.md"
requires-python = ">=3.10"

dependencies = [
    "Django>=5.0"
]
```

Choose supported versions according to your project's compatibility requirements.

15.3 Package Structure

```text
my_django_utils/
│
├── src/
│   └── my_django_utils/
│       ├── __init__.py
│       └── helpers.py
│
├── tests/
│
├── README.md
├── LICENSE
└── pyproject.toml
```

15.4 Build Tools

Common packaging tools:

- setuptools
- Hatchling
- Flit
- Poetry
- build

Install build:

```bash
pip install build
```

15.5 Build a Distribution

```bash
```

```
python -m build
```

This creates distribution files inside `dist/`.

Common outputs:

- `.whl` — Wheel distribution
- `.tar.gz` — Source distribution

15.6 Validate Package Metadata

Install Twine:

```bash
pip install twine
```

Check:

```bash
twine check dist/*
```

15.7 Install Local Package

```bash
pip install .
```

Editable installation:

```bash
pip install -e .
```

15.8 Publish to TestPyPI

Upload:

```bash
twine upload --repository testpypi dist/*
```

Test installation:

```bash
pip install \
    --index-url https://test.pypi.org/simple/ \
    my-django-utils
```

15.9 Publish to PyPI

```bash
twine upload dist/*
```

Use an appropriate PyPI API token.

15.10 Semantic Versioning

Format:

```text
MAJOR.MINOR.PATCH
```

- MAJOR: Breaking changes
- MINOR: Backward-compatible features
- PATCH: Bug fixes

15.11 README and License

README should explain:

- What the package does
- Installation
- Usage
- Configuration
- Examples
- Compatibility
- License

A license defines how others may use and distribute the software.

15.12 Package Testing

Test:

- Installation
- Imports
- Core functions
- Compatibility
- Dependencies
- Distribution contents

---

16. Git and Development Workflow

16.1 What is Git?

Git is a distributed version control system used to track source code changes.

GitHub is a platform for hosting Git repositories and collaborating.

16.2 Initialize Git

```bash
git init
```

16.3 Clone Repository

```bash
git clone https://github.com/username/repository.git
```

16.4 Check Status

```bash
git status
```

16.5 Add Files

```bash
```

```
git add .
```

Add a specific file:

```bash
git add models.py
```

16.6 Commit

```bash
git commit -m "Add student model"
```

16.7 Push

```bash
git push origin main
```

16.8 Pull

```bash
git pull origin main
```

16.9 Branches

Create:

```bash
git branch feature-student-api
```

Switch:

```bash
git switch feature-student-api
```

Create and switch:

```bash
git switch -c feature-student-api
```

16.10 Merge

```bash
git switch main

git merge feature-student-api
```

16.11 Pull Requests

A Pull Request allows developers to propose changes and request code review before merging.

Typical workflow:

1. Create a branch.
2. Make changes.

3. Commit changes.
4. Push branch.
5. Open Pull Request.
6. Review code.
7. Fix requested changes.
8. Merge.

16.12 .gitignore

A `.gitignore` file tells Git which files to ignore.

Example:

```gitignore
venv/
.env
__pycache__/
*.pyc
db.sqlite3
media/
staticfiles/
.pytest_cache/
.coverage
```

Do not ignore files that your project needs to build or run.

16.13 Requirements and Dependency Management

Save dependencies:

```bash
pip freeze > requirements.txt
```

Install:

```bash
pip install -r requirements.txt
```

For more advanced dependency management, consider Poetry, uv, or another modern package manager.

16.14 Code Formatting and Linting

Common tools:

- Black
- Ruff
- Flake8
- isort
- mypy

Install Ruff:

```bash
pip install ruff
```

Check code:

```bash
ruff check .
```

```

Format:

```bash
ruff format .
```

16.15 PEP 8

PEP 8 is the Python style guide.

Examples:

- Use meaningful names.
- Use four spaces for indentation.
- Keep code readable.
- Use lowercase_with_underscores for functions and variables.
- Use CapWords for classes.
- Avoid unnecessary complexity.

16.16 Documentation

Document:

- Project setup
- Installation
- Environment variables
- Database configuration
- API endpoints
- Testing
- Deployment
- Contribution guidelines

---

17. Interview Preparation

17.1 Explain Django MVT

Django uses Model-View-Template architecture.

- Model manages data.
- View handles application logic.
- Template handles presentation.

17.2 Explain Request-Response Lifecycle

Request passes through middleware, URL resolver, view, model/database, and response processing before returning to the client.

17.3 Project vs App

A project contains the overall configuration. An app is a reusable module for a particular functionality.

17.4 Middleware and Execution Order

Middleware processes requests and responses globally. Request processing follows configured order; response processing travels back through the middleware stack.

17.5 Models and Migrations

Models define database structure. Migrations record and apply schema changes.
```

17.6 QuerySet Lazy Evaluation

QuerySets delay database execution until their results are evaluated.

17.7 select_related vs prefetch_related

- `select_related()` uses SQL JOIN for ForeignKey and OneToOne.
- `prefetch_related()` performs separate queries for ManyToMany and reverse relationships.

17.8 ForeignKey vs OneToOneField vs ManyToManyField

- ForeignKey: Many-to-one.
- OneToOneField: One-to-one.
- ManyToManyField: Many-to-many.

17.9 get() vs filter()

`get()` returns one object or raises an exception.

`filter()` returns a QuerySet.

17.10 annotate() vs aggregate()

`annotate()` adds calculated values to individual objects or groups.

`aggregate()` calculates a summary for the QuerySet.

17.11 Q Objects vs F Expressions

- Q objects build complex query conditions.
- F expressions reference database fields in queries and updates.

17.12 Signals

Signals allow components to respond to events such as model saves or deletes.

Use them for decoupled reactions, not as a replacement for all business logic.

17.13 Authentication vs Authorization

Authentication verifies identity.

Authorization checks access rights.

17.14 FBV vs CBV

FBV uses functions.

CBV uses classes, inheritance, and mixins.

17.15 APIView vs ViewSet

APIView provides explicit HTTP method handlers.

ViewSet groups related API actions and can use routers to generate URLs.

17.16 Serializer Validation

Serializers validate incoming data before creating or updating objects.

17.17 Transactions and Atomicity

Transactions group database operations so they succeed or fail together.

`transaction.atomic()` provides an atomic transaction block.

17.18 Database Indexes

Indexes help speed up database searches but add storage and write overhead.

17.19 CSRF vs CORS

CSRF is an attack involving unwanted requests using a user's authenticated browser context.

CORS is a browser mechanism that controls cross-origin access.

They solve different problems.

17.20 Django Security Practices

- Use HTTPS.
- Set DEBUG=False.
- Protect secrets.
- Validate input.
- Use ORM safely.
- Apply permissions.
- Validate file uploads.
- Update dependencies.
- Use secure cookies.

17.21 Deployment Checklist

- Production settings configured.
- Environment variables configured.
- DEBUG disabled.
- ALLOWED_HOSTS configured.
- Database configured.
- Migrations applied.
- Static files collected.
- Media storage configured.
- HTTPS enabled.
- Logging enabled.
- Backups configured.
- Deployment tested.

More Interview Questions

1. What is Django ORM?
2. What is `class Meta`?
3. What is `db_table`?
4. What is `managed=False`?
5. What is an abstract model?
6. What is a proxy model?
7. What is a custom manager?
8. What is a database router?
9. What is a data migration?
10. What is `inspectdb`?
11. What is `transaction.atomic()`?
12. What is `bulk_create()`?
13. What is a serializer?
14. What is ModelSerializer?
15. What is JWT?
16. What is API throttling?
17. What is API versioning?
18. What is CORS?

19. What is a reverse proxy?
20. What is Python packaging?

---

18. Practice Projects

18.1 Blog Application

Features:

- User registration
- Login and logout
- Create posts
- Read posts
- Update posts
- Delete posts
- Categories
- Comments
- Authentication
- Admin panel

Concepts:

- Models
- Relationships
- Forms
- Views
- Templates
- Authentication
- ORM

18.2 Student Management System

Features:

- Add students
- Update students
- Delete students
- Student details
- Department management
- Search
- Filtering
- Admin panel

Concepts:

- CRUD
- ForeignKey
- ModelForm
- QuerySets
- Admin customization

18.3 Task Management REST API

Features:

- User registration
- Login
- Create tasks
- Update tasks
- Delete tasks
- List tasks
- Task status

- Priority
- Pagination
- Filtering

Concepts:

- DRF
- Serializer
- ViewSet
- Router
- JWT
- Permissions

18.4 E-commerce API

Features:

- Users
- Products
- Categories
- Cart
- Orders
- Payments
- Inventory
- Reviews

Concepts:

- ManyToMany
- ForeignKey
- Transactions
- Nested serializers
- Authentication
- Permissions
- API design

18.5 JWT-Protected User API

Features:

- Registration
- Login
- Access token
- Refresh token
- User profile
- Protected endpoints
- Permission checks

18.6 Reusable Django Package

Features:

- Reusable utility functions
- Package structure
- Tests
- Documentation
- Versioning
- Build distribution
- Local installation

---

19. Task Management REST API Capstone

19.1 Project Overview

Build a professional Task Management REST API using Django, DRF, PostgreSQL, and JWT.

19.2 Technology Stack

- Python
- Django
- Django REST Framework
- PostgreSQL
- Simple JWT
- django-filter
- drf-spectacular
- Git and GitHub

19.3 Project Structure

```text
task_management/
│
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── users/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── urls.py
│   └── tests.py
│
├── tasks/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── permissions.py
│   ├── filters.py
│   ├── urls.py
│   └── tests.py
│
├── manage.py
├── requirements.txt
└── .env
```

19.4 Task Model

```python
from django.conf import settings
from django.db import models

class Task(models.Model):

    STATUS_CHOICES = [
        ("pending", "Pending"),
        ("in_progress", "In Progress"),
        ("completed", "Completed"),
    ]

    PRIORITY_CHOICES = [
        ("low", "Low"),
```

```
        ("medium", "Medium"),
        ("high", "High"),
    ]

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="tasks"
    )

    title = models.CharField(max_length=200)

    description = models.TextField(blank=True)

    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default="pending"
    )

    priority = models.CharField(
        max_length=20,
        choices=PRIORITY_CHOICES,
        default="medium"
    )

    due_date = models.DateField(
        null=True,
        blank=True
    )

    created_at = models.DateTimeField(auto_now_add=True)

    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title

    class Meta:
        ordering = ["-created_at"]
        indexes = [
            models.Index(fields=["user", "status"]),
            models.Index(fields=["due_date"]),
        ]
```

19.5 Task Serializer

```python
from rest_framework import serializers
from .models import Task

class TaskSerializer(serializers.ModelSerializer):

    class Meta:
        model = Task

        fields = [
            "id",
            "title",
            "description",
            "status",
            "priority",
            "due_date",
```

```
            "created_at",
            "updated_at",
        ]

        read_only_fields = [
            "id",
            "created_at",
            "updated_at",
        ]
```

19.6 Task Permission

Users must access only their own tasks.

```python
from rest_framework.permissions import BasePermission

class IsTaskOwner(BasePermission):

    def has_object_permission(
        self,
        request,
        view,
        obj
    ):

        return obj.user == request.user
```

19.7 Task ViewSet

```python
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated

from .models import Task
from .serializers import TaskSerializer
from .permissions import IsTaskOwner

class TaskViewSet(viewsets.ModelViewSet):

    serializer_class = TaskSerializer

    permission_classes = [
        IsAuthenticated,
        IsTaskOwner
    ]

    def get_queryset(self):

        return Task.objects.filter(
            user=self.request.user
        )

    def perform_create(self, serializer):

        serializer.save(
            user=self.request.user
        )
```

Filtering the queryset by the authenticated user is important. Object-level permissions alone do not filter list responses.

19.8 Task Router

```python
from rest_framework.routers import DefaultRouter
from .views import TaskViewSet

router = DefaultRouter()

router.register(
    "tasks",
    TaskViewSet,
    basename="task"
)

urlpatterns = router.urls
```

19.9 API Endpoints

Method	Endpoint	Purpose
POST	`/api/token/`	Login
POST	`/api/token/refresh/`	Refresh token
GET	`/api/tasks/`	List tasks
POST	`/api/tasks/`	Create task
GET	`/api/tasks/1/`	Retrieve task
PUT	`/api/tasks/1/`	Full update
PATCH	`/api/tasks/1/`	Partial update
DELETE	`/api/tasks/1/`	Delete task

19.10 Capstone Requirements

- User registration and login
- JWT authentication
- Task CRUD
- Each user accesses only their own tasks
- Status management
- Priority management
- Due date
- Filtering
- Searching
- Ordering
- Pagination
- Serializer validation
- Permission tests
- API documentation
- Environment-based settings
- PostgreSQL configuration
- Deployment
- README documentation

19.11 Capstone Testing Checklist

- Test user registration.
- Test login.
- Test invalid credentials.
- Test task creation.
- Test task listing.
- Test task update.
- Test task deletion.
- Test unauthenticated requests.
- Test user A cannot access user B's tasks.
- Test filtering.

- Test pagination.
- Test validation errors.

---

20. Learning Roadmap

Follow this order to become a Django Backend Developer.

Phase 1: Python Fundamentals

Study:

- Variables
- Data types
- Conditions
- Loops
- Functions
- Lists
- Tuples
- Sets
- Dictionaries
- OOP
- Exception handling
- Modules and packages
- Decorators
- File handling

Practice: 30–50 Python problems.

Phase 2: HTTP and Web Basics

Study:

- Client and server
- HTTP
- HTTPS
- Request and response
- HTTP methods
- Status codes
- JSON
- REST API
- Cookies
- Sessions

Practice: Understand how a browser communicates with a server.

Phase 3: Django Fundamentals

Study:

- Django installation
- Project and app
- Settings
- URL routing
- Views
- Templates
- Middleware
- MVT architecture

Practice: Build a simple website.

Phase 4: Models, Migrations, and ORM

Study:

- Model fields
- Relationships
- `class Meta`
- Migrations
- QuerySets
- Q and F expressions
- Aggregation
- Annotation
- Transactions
- Query optimization

Practice: Student Management System.

Phase 5: Forms and Authentication

Study:

- Forms
- ModelForms
- Validation
- User model
- Login and logout
- Permissions
- Groups
- Sessions
- Custom user model

Practice: Blog application with authentication.

Phase 6: Django REST Framework

Study:

- Serializers
- APIView
- GenericAPIView
- Mixins
- ViewSets
- Routers
- CRUD APIs
- JWT
- Permissions
- Pagination
- Filtering
- Throttling
- API documentation

Practice: Task Management REST API.

Phase 7: Testing and Security

Study:

- TestCase
- API testing
- Mocking
- Logging
- CSRF
- XSS
- SQL injection
- CORS
- Secure settings

- File upload validation

Practice: Write tests for every important API.

Phase 8: Query Optimization

Study:

- N+1 queries
- select_related
- prefetch_related
- Indexes
- Caching
- Transactions
- Bulk operations
- Profiling

Practice: Optimize an existing Django application.

Phase 9: Deployment

Study:

- Production settings
- Environment variables
- PostgreSQL
- Gunicorn
- Uvicorn
- Nginx
- Static files
- Media storage
- HTTPS
- Logging
- Backups

Practice: Deploy a complete API.

Phase 10: Portfolio and Interview

Build:

1. Blog Application
2. Student Management System
3. Task Management REST API
4. E-commerce REST API
5. Reusable Django Package

Prepare:

- Django fundamentals
- Python OOP
- SQL and PostgreSQL
- Django ORM
- DRF
- API security
- Git
- Testing
- Deployment
- DSA and problem-solving

Official Documentation

1. Django Documentation: https://docs.djangoproject.com/
2. Django REST Framework: https://www.django-rest-framework.org/
3. Python Documentation: https://docs.python.org/3/
4. Python Packaging: https://packaging.python.org/
5. PostgreSQL Documentation: https://www.postgresql.org/docs/
6. Git Documentation: https://git-scm.com/doc
